Project Title: Internship Management System API

Framework: NestJS + TypeScript

Database: PostgreSQL + TypeORM

1. Project Overview:

The **Internship Management System API (IMS Backend)** is a secure RESTful backend application designed to manage internship opportunities between **students and companies**. The platform allows **companies to post internship opportunities**, **students to apply**, and **administrators to monitor and manage the system**.

- **The system focuses on the following functionalities:**

  1. User management (students, companies, admin)
  2. Internship posting and management
  3. Student internship applications
  4. Application review and status tracking

The backend will be implemented using **NestJS with TypeScript**, and all endpoints will be tested through **Postman and Swagger UI**. No frontend interface is required.

2. Core Functional Requirements:

#	Feature	HTTP Method	Endpoint	Description	Priority
1	User Registration & Login	POST	/auth/register /auth/login	Register with role (student/company/admin), login returns JWT	Must
2	Get Current User Profile	GET	/auth/me	Returns authenticated user details	Must
3	Company Creates Internship	POST	/internships	Company posts internship (title, company name, description, location, deadline)	Must
4	List All Internships	GET	/internships	Students view all available internships	Must
5	Get Single Internship	GET	/internships/:id	Get detailed internship information	Must

6	Student Applies for Internship	POST	/applications	Student submits application with resume link	Must
7	View My Applications	GET	/applications/my	Student views their applied internships	Must
8	Company Views Applicants	GET	/internships/:id/applicants	Company sees all applicants for their internship	Must
9	Update Application Status	PATCH	/applications/:id/status	Company approves or rejects applications	Must
10	Withdraw Application	DELETE	/applications/:id	Student withdraws their application	Should
11	Admin Manage Users	GET/DELETE	/admin/users	Admin views or removes users	Should

3. Database Entities & Relationships

The system will implement multiple database relationships including **One-to-Many**, **Many-to-One**, and **Many-to-Many** relationships.

Entity	Key Fields	Relationships Implemented	Type
User	id, email, password (hashed), fullName, role (student/company/admin), studentId (optional), companyName (optional), department	One Company (User) → Many Internships (as internship creator) One Student (User) → Many Applications	One-to-Many
Internship	id, title, companyName, description, location, deadline, createdBy (User)	Many Internships ← One Company (User) Many Internships ↔ Many Students via Application	Many-to-One Many-to-Many
Application	id, student (User), internship (Internship), resumeLink, status (Pending / Accepted / Rejected), appliedAt	Many Applications ← One Student (User) Many Applications ← One Internship	Many-to-One

❖ **Required Relationship Types Demonstrated**

- **One-to-Many / Many-to-One:** A company can post multiple internships.
- **Many-to-Many:** A student can apply to multiple internships, and an internship can receive applications from multiple students.

4. Email Notification (Mailer):

⇨ Email notifications will be implemented using:

- @nestjs-modules/mailer
- Nodemailer

❖ **Email Triggers:**

Action	Trigger	Recipient	Subject	Content
Student applies for internship	After application submission	Student	Application Submitted	Internship title and confirmation
Application accepted/rejected	After company decision	Student	Application Status Update	Acceptance or rejection message
New application received	After student applies	Company	New Internship Application	Applicant information

❖ **Technical Requirements for Mailer:**

- SMTP credentials stored in **.env**
- Development testing using **Mailtrap or Ethereal**
- Template engine (**Handlebars preferred**)
- Email templates stored in: `src/mail/templates/`

5. Technical Requirements:

Category	Requirement	Mandatory
Language	TypeScript	Yes
Framework	NestJS	Yes
Authentication	JWT + Passport	Yes
Authorization	Role-based Guards	Yes
Database	PostgreSQL	Yes
ORM	TypeORM	Yes
Relationships	OneToMany / ManyToMany	Yes
Input Validation	class-validator + class-transformer	Yes
API Documentation	Swagger UI	Yes
Email Integration	Mailer + Nodemailer	Yes
Password Security	bcryptjs hashing	Yes
Error Handling	HttpException + global filters	Yes
Configuration	.env using @nestjs/config	Recommended
Testing	Jest unit tests	Recommended
Version Control	Git + GitHub	Yes